



Evgenii Meshcheriakov

Climate Chamber Control System Automation Using PLC

Metropolia University of Applied Sciences

Bachelor of Engineering

Information Technology

Bachelor's Thesis

12 November 2024

Abstract

Author:	Evgenii Meshcheriakov
Title:	Climate Chamber Control System Automation Using PLC
Number of Pages:	40 pages
Date:	12 November 2024
Degree:	Bachelor of Engineering
Degree Programme:	Information Technology
Professional Major:	Smart IoT Systems
Supervisors:	Keijo Lämsikunnas, Senior Lecturer

The primary objective of this final year project was to design and construct a control system for a climate chamber which is fully and automatically operated with the use of a Siemens Programmable Logic Controller (PLC). The project was given to the UrbanFarmLab of Metropolia University of Applied Sciences (UAS) with the aim of improving the accuracy and repeatability factors of the environmental conditions inside the chamber, which for the purposes of the UAS is especially important.

The thesis presents the design and programming of a PLC based system that includes sensors and actuators to control the following system variables: temperature, humidity, level of CO₂ and lighting. The implemented solutions included interfacing devices such as a Modbus communication protocol, time schedule-oriented algorithms and error management subsystems for the active protection of the integrated system. The Siemens TIA Portal was the included software for the configuration and programming of the PLC.

The self-contained climate chamber control system has been completed and it supports two modes of operation: manual and automatic. The designed system is able to keep within the preset range of environmental parameters with little input from the user, and it allows monitoring the process through a user-friendly interface. Suggested improvements for the next versions of the system development are adding more sensors, adding an NTP time server for time synchronization and improving the data logging features of the system.

This project offers a complete approach to climate chamber automaton and serves as a preview into greater possibilities in the rollout of vertical farming and preservation systems respectively.

Keywords: PLC, climate control, Modbus, urban farming, Siemens, control system, automation

The originality of this thesis has been checked using Turnitin Originality Check service.

Contents

List of Abbreviations

1	Introduction	1
2	Differences Between Siemens PLC and Ordinary Programming	3
2.1	Environment and Tools	3
2.2	Programming Paradigms	4
2.3	Execution and Performance	5
2.4	Reliability and Safety	5
3	System Requirements and Components	6
3.1	Specification of Climate Chamber Conditions	6
3.2	Siemens PLC and Modules	7
3.3	Sensors	8
3.4	Actuators	9
3.5	Dehumidifier Operation Limits	10
4	Implementation	11
4.1	Structure	12
4.2	Modes	13
4.2.1	Off Mode	13
4.2.2	Automatic Mode	14
4.2.3	Manual Mode	18
4.3	Schedule	19
4.3.1	Scheduling Logic	20
4.3.2	Value_from_schedule Function Block	21
4.3.3	Interpolate Function Block	21
4.4	Implementing Modbus Communication in PLC	22
4.4.1	Setting up and Read and Write Operations	22
4.4.2	Modbus Data Variables	27
4.5	Error Handling	27
4.6	Safety Measures	28
4.7	User Interface	29
4.7.1	Screen Home	29
4.7.2	Screen State	30

4.7.3	Screen Schedule	31
4.7.4	Screen Errors	31
4.7.5	User Input Validation and Error Prevention	32
5	Future Enhancements and Recommendations	34
5.1	Data Logging	34
5.2	Time Synchronization	35
5.3	Runtime Errors	36
5.3.1	Air Conditioner Communication Error	36
5.3.2	Too Many Tags	36
6	Conclusion	36
	References	39

List of Abbreviations

AI	Analogue Input
CPU	Central Processing Unit
DC	Direct Current
DI	Digital Input
DO	Digital Output
I/O	Input Output
IDE	Integrated Development Environment
LAD	Ladder Logic
LSB	Least Significant Bits
OB	Organization Blocks
PLC	Programmable Logic Controller
RDI&E	Research, Development, Innovation, and Education
SCL	Structured Control Language
TIA Portal	Totally Integrated Automation Portal
UAS	University of Applied Sciences
VMD	Vapor Mass Deficit

1 Introduction

In a world where cities are growing rapidly and sustainability is more important than ever, urban farming is stepping up as a promising solution to the rising demand for fresh, locally sourced food [1]. Metropolia University of Applied Sciences (from here on Metropolia), in partnership with the Helsinki region, has laid the foundation for a dynamic ecosystem focused on urban indoor food production and new technologies. At the heart of this initiative is Metropolia's dedication to research, development, innovation, and education (RDI&E) in urban farming technologies, which is embodied in the Metropolia UrbanFarmLab – a platform that is leading the way in developing urban indoor farming systems.

With a common vision of developing and advancing an environmentally clean solution, Metropolia has welcomed the idea of introducing advanced technologies in urban farming. This progressive thinking has paved way for an advanced control system for climate chambers, which are necessitated in providing optimal growth and experimentation conditions for the plants. Climate chambers are important for controlled environment studies allowing the performance of various studies that demand tight control of temperature, humidity and carbon dioxide (CO₂) levels. [2.]

The research and development objectives of the final year project focus primarily on the intriguing activities at the Metropolia UrbanFarmLab. Thus, this work addresses the issue of automating the control systems of climate chambers by means of Programmable Logic Controller (PLC). PLCs are widely used in industrial automation, and they are essential in the control of climate conditions inside the climate chamber during plant production experiments and applications.

The deployment of a PLC-based control system offers great improvement advantages such as enhanced accuracy, improvement of work efficiency, and a

high amount of data for analytics. This system minimizes manual control and is ideal for the automated handling of the climate chamber. This in-built capability of the PLC incorporates acquisition, transmission, processing and output of information effectively leading to control and reporting. Therefore, there are no variations caused, and the results can be said to be reliable. Hence, the system aids in the achievement of one big objective which is urban agriculture and sustainable farming systems.

Within the scope of this project, the main activity was designing a good logic for the PLC, which is responsible for the workload supervision system. This logic has to perform satisfactorily and integrate well with sensor and actuator elements responsible for temperature, humidity, and carbon dioxide regulation.

Along with the focus on the programming of the PLC unit, this project widened the focus to incorporate the importance of proper integration of the sensorial and actuating systems plus the communication system based on the Modbus communication standard. While data from important sensors is available, actuators in the system, such as the heat pump, the humidifier, the dehumidifier, and the CO₂ valve, carry out their tasks or actions. These systems are connected using the Modbus protocol and basic analogue signals.

This work turned out to be one major step in the Metropolia UrbanFarmLab project. Hence, the intent of this project was to appreciate and make an effort in the attainment of the particular objectives and goals related to the project. The impact of the results of this study goes beyond the bounds of theories in the classroom. They include practical measures, raise awareness, and create business linkages. It is important to recognize the possibilities the system of automation puts across and the benefits that come along with easing the pressure in the production of food sustainably.

Subsequent sections cover the process of the development of PLC Logic, its communication with sensors and actuators, user interface capabilities and perspectives of further improvement. First, the difference between conventional programming and PLC programming is presented.

2 Differences Between Siemens PLC and Ordinary Programming

The main purpose of Siemens PLC programming is to directly control and automate industrial processes and machines. The aim is to make sure that the actual generating or producing activity is carried out in the most efficient manner. This programming, for example, is most commonly found in factories, where it helps in the operation of machines, assembly lines, and other automated activities. [3.]

Conversely, languages such as C or C++ that are not limited to any one domain are much more expansive. They may be used for an extensive range of software applications, as they are not limited to a particular sector. Languages such as these are highly flexible, enabling them to be applied in many areas other than industrial automation. For example, they are generally used for building up systems, developing desktop programs, designing games, creating embedded applications, and much more.

The next section will address the most important differences and the similarities between these two programming methods.

2.1 Environment and Tools

Siemens PLC programming engages the users in using certain development environments and utilities, such as Siemens TIA Portal (Totally Integrated Automation Portal). This application performs the controlling and organizational purposes for a user who aims to develop specifically a Siemens PLC application. In this context, the engineers can also use graphically oriented programming languages, namely Ladder Logic (LAD), Function Block Diagrams (FBD), and Sequential Function Charts (SFC) for programming. These languages are constructional and easy to use, particularly for engineers who have prior knowledge of electrical diagrams and flow charts.

On the other hand, programming in languages such as C or C++ is usually done in an Integrated Development Environment (IDE), such as Visual Studio, Eclipse, or Visual Studio Code. These platforms include many functions of coding, debugging, and compiling and running programs. Each of these tools allows the programmer to edit text in a window with the knowledge that this will eventually be converted to executable, machine-specific code by an intermediate piece of hardware or program known as a compiler. Furthermore, these environments also include but are not limited to management of projects through debugging and versioning aids for programmers using those environments.

2.2 Programming Paradigms

PLC programming is done primarily in a control-oriented fashion. This implies that it is mainly concerned with the writing of functional programs that control the inputs and outputs of the machinery to within real time limits. Five programming languages are specified in the standard for programming of PLCs in IEC 61131-3. In addition to LAD, FBD, ST discussed earlier, there is also Instruction List (IL) and Sequential Function Chart (SFC). However, IL and SFC are less popular than the first three and were not used in the project for that matter at all.

To start with, Ladder Logic is quite graphical in nature and very similar to electrical relay logic diagrams which makes it easy for engineers and technicians to comprehend and work with the logic.

C and C++ can incorporate several modes of programming including procedural and object orientated programming. These languages are overhead optimizing so that system resources are used as effectively as possible in working up complex data structures and algorithms and system software. Another thing is that they are based on text which gives the user no option but to understand advanced programming concepts such as the memory management and the hidden logic of computing.

2.3 Execution and Performance

PLCs are designed for continuous operation, and thus, control functions must be carried out within given time boundaries. Siemens PLCs, among others, are particularly tuned to operate in a deterministic manner, which makes sure processes are controlled with extreme accuracy and dependability. The execution of a PLC program is done to a cyclic type, where the program reads the inputs, processes the logic and changes the output continuously in a loop, every 100 ms, for example.

Programming languages such as C and C++ are general-use in that they can be used in many applications and not for real-time systems in the first place even if with a moderate modification they can be used for that purpose. C and C++ languages and their accompanying compilers are mostly developed around event or process control attributes. The performance of such an application is also determined by the performance of the code itself and the performance of the code's execution environment (hardware). These programming languages are oriented towards taking advantage of the physical features of the target system which explains their use in real time systems, but they also need to be tamed so that they meet hard timings. [4, p. 153-160]

2.4 Reliability and Safety

Siemens PLCs have been developed with a high degree of emphasis on reliability and safety, which is essential in industrial environments. The programming of these PLCs contains elements that facilitate fault detection and error control, which are meant to help operations run smoothly even under challenging situations. There is a tendency from engineers to incorporate redundancy and robustness for better performance in the design of PLC systems.

However, this is not the case when developing fault-tolerant systems with general-purpose programming languages. Such systems can be implemented,

but they do not provide the same PLC's built-in safety mechanisms. Thus, the developers have to implement the error handling and the fault tolerance of the application on their own, making the implementation difficult and error-prone.

To conclude, there is a clear distinction between the objectives directed towards encouraging the programming of Siemens PLCs and other conventional languages. A PLC is designed with respect to principles of control and automation of production processes. It is a powerful and very easy tool for dealing with time dependent processes. On the one hand, C and C++ are more suited for a variety of software development applications, but on the other hand they require much more expertise and management [4, p. 104-109] to achieve similar reliability and real-time performance.

3 System Requirements and Components

There are specific system requirements and components that must be implemented and used in order to accomplish the required level of automation of climate chamber control systems. This section lays out the important aspects that are needed to achieve precise control of the climate condition in the chamber.

3.1 Specification of Climate Chamber Conditions

The environment within the chamber is crucial for plant development and needs to be controlled accurately. The optimal temperature, humidity, and light levels will differ according to the kinds of plants under cultivation. For this project, strawberries were used as the model plant, given their popularity and specific climate needs. Table 1 outlines the initial schedule and conditions for the climate chamber.

Table 1. Initial climate chamber schedule and conditions.

Parameter	Daytime (08:00 - 00:00)	Nighttime (00:00 - 08:00)
-----------	-------------------------	---------------------------

Temperature (C°)	22	8
Relative humidity (%)	75	60
CO ₂ (ppm)	1000	0
Light	ON	OFF

This schedule in Table 1 is designed to simulate natural day and night cycles, promoting the healthy growth and development of the plants.

3.2 Siemens PLC and Modules

The core of the control system is the Siemens PLC, or technically speaking the SIMATIC S7-1200, CPU 1215C, compact CPU, DC/DC/DC model.

As per a Siemens datasheet [5], it is equipped with two PROFINET ports and onboard I/O capabilities, including:

- 14 digital inputs (DI) at 24 V DC
- 10 digital outputs (DO) at 24 V DC, 0.5 A
- Analogue inputs (AI) at 0-10 V DC
- 2 analogue outputs (AO) at 0-20 mA DC

The PLC is powered by a DC supply of 20.4-28.8 V and features a program/data memory of 200 KB, making it suitable for the complex control tasks required in this project.

Additionally, RS485 communication board CB 1241 with an integrated Modbus RTU master and a slave protocol was installed to provide a Modbus connection.

3.3 Sensors

To monitor and maintain the desired environmental conditions, the Siemens QPA2062 all-in-one room air quality sensor was selected. The sensor measures temperature, relative humidity, and CO₂ concentration and is connected as shown in Figure 1.

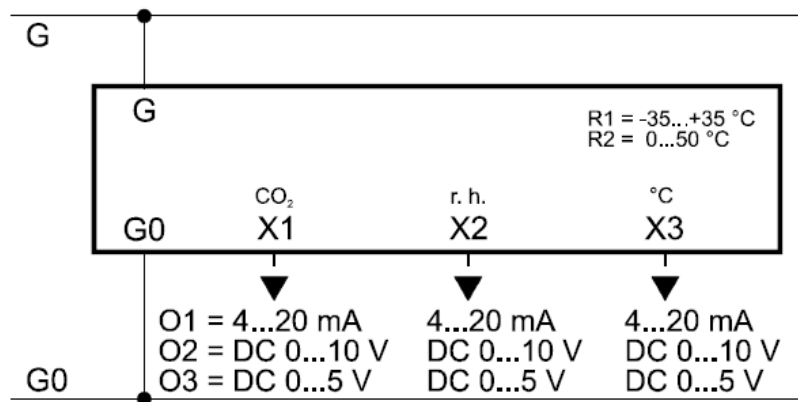


Figure 1. Connections diagram [6].

As illustrated in the Figure 1, there are different modes of output. Option O1 = 4...20 mA was selected as the most reliable and suitable type of the output for the application.

The connection and possible output are described below [6]:

- G system potential AC 24 V (SELV) or DC 15 - 35 V
- G0 system neutral and measuring neutral
- X1 – CO₂ gas concentration (0 - 2000 ppm)
- X2 – relative humidity (0...100%)
- X3 – temperature. Temperature range R2 (0 - 50 °C) was selected since a negative temperature cannot be reached with the selected air conditioner and it shall not be set as a target temperature.

3.4 Actuators

There are several actuators in the system: air conditioner (sometimes in the document it can also be called “heat pump”), humidifier with its own fan, dehumidifier, CO₂ valve and LED light. These are described below.

- **Air Conditioner:** comprised of an outdoor unit (PACI ELITE U-71PZH4E8) and an indoor unit (S-6010PK3E); it maintains the desired temperature and manages humidity by condensing excess moisture. The air conditioner is controlled via Modbus. [7]
- **Humidifier and Fan:** an ultrasonic model Mist Maker 5 (DK5); both the humidifier and its fan are relay-controlled to maintain the required humidity levels.
- **Dehumidifier:** an industrial model Condair DC 50C, with a built-in Modbus, ensures excess moisture is removed efficiently. The model is not equipped with a humidity sensor, so it cannot be controlled via Modbus.
- **CO₂ valve:** regulates the CO₂ concentration within the chamber. CO₂ control is a critical component of the climate chamber's operation, as it directly influences plant growth and photosynthesis.
- **LED light:** light control is an essential aspect of the climate chamber's functionality, as it directly affects plant growth, development, and overall health. The system is designed to manage lighting based on a predefined schedule or manual override, allowing for flexibility in meeting the specific light requirements of different plant species.

Conditioned air is circulated by industrial fans through a duct system although this element is outside the scope of the final year project.

Each of these components plays a vital role in maintaining precise climate conditions needed for the plants to thrive. The integration of these sensors and actuators with the PLC and the Modbus communication protocol enables automated control, ensuring that the environment within the climate chamber remains stable and conducive to plant growth.

3.5 Dehumidifier Operation Limits

The device has an operation limit (see Figure 2) outside of which it must not be used.

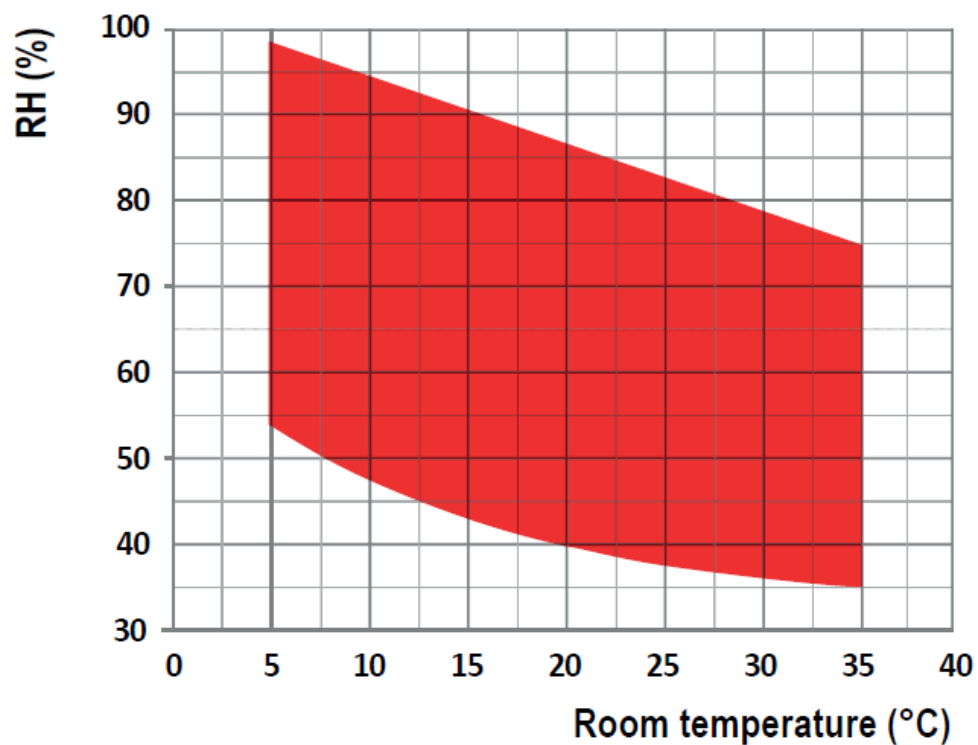


Figure 2. Dehumidifier operation limits. [8].

In order to ensure safe operation of the condenser, Figure 2 above has been modelled by the four equations.

The curved bottom line is represented by the following cubic equation:

$$y_1 = -0.000691358 x^3 + 0.0614815x^2 - 2.10741 x + 63.0864$$

On other hand, the top linear line is represented by the following simple linear equation:

$$y_2 = -0.75 x + 101.25,$$

Where:

y_1 and y_2 are the limits for humidity and x is the current temperature.

The left and right lines are just simple temperature limits represented by lines $x_1 = 5$ and $x_2 = 35$.

4 Implementation

The implementation of the climate chamber control system using a Siemens PLC involves various components and programming structures to ensure precise and reliable control over the environmental conditions within the chamber. This section details the key elements of the implementation process.

Among the five block types available on the PLC, three were employed in the implementation: organization blocks, which establish the logical structure; functions, which encapsulate routines for repetitive tasks such as scheduling and value interpolation; and instance data blocks, which serve as repositories for data storage.

Two Organization Blocks (OBs) are utilized within the program, serving as the interface between the operating system and the control system software. These OBs are invoked by the operating system and, in the context of the current software, manage cyclic program execution and interrupt-driven processes.

The program logic utilizes two programming languages: SCL, a high-level language derived from PASCAL, and LAD, a graphical language that visually represents logic through circuit diagram-based symbols.

4.1 Structure

The controller's structure is straightforward, as depicted in Figure 3.

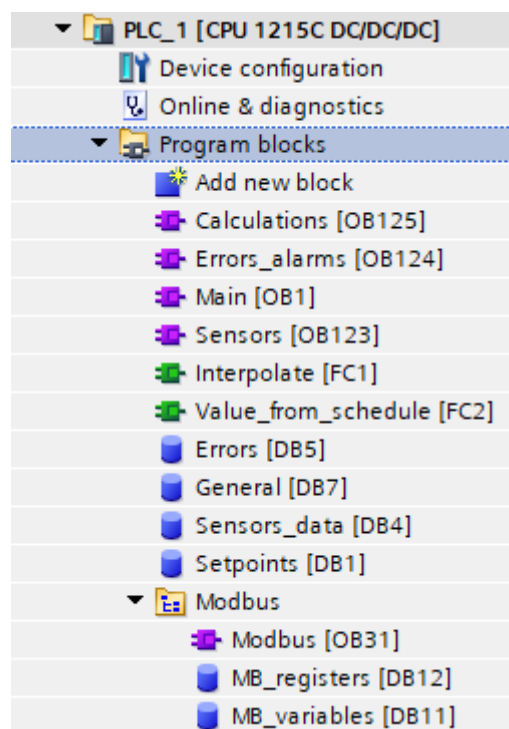


Figure 3. Files tree.

The main components of the program are organized into files that handle different aspects of the control system. As it seen in Figure 3, these files include the following:

- Calculations
- Errors_alarms
- Main
- Sensors

These files are defined as a program cycle and are executed on every PLC cycle. Modbus communication and associated data variables are located in the Modbus folder, which is defined as a cyclic interrupt with a 500 ms cycle. Additional files include data blocks such as Errors, General, Sensors_data, and Setpoints, as well as functions for interpolation and scheduling.

4.2 Modes

The control system operates in three modes, which are defined by the variable Setpoints.mode. The modes are selected based on the two least significant bits (LSB) of this variable, with the following decimal representations:

- 0: Off mode
- 1: Automatic mode
- 3: Manual mode

Bit 0 is set and reset in the UI, in the home screen via the main switch button. Bit 1 of the variable is flipped via the neighbor button on the same screen (see Section 4.7.1).

4.2.1 Off Mode

In Off mode, all actuators are turned off to prevent accidental activation. The corresponding PLC tags and Boolean variables are set to false. The code for deactivating the actuators is as follows:

- Power off the AC: `"MB_registers".AC_command[#AC_comm.On] := 0;`
- Stop humidifier fan: `"Setpoints".Manual_humidifier_fan_speed := 0;`
`"General".Fan_speed := 0;`
- Stop dehumidifier: `"Q0.0 A2 Relay 1 A1" := false;`
`"General"."Dehumidifier 1" := false;`
- Stop humidifier: `"Q0.1 A2 Relay 2 A1" := false;`
`"General"."Humidifier 1" := false;`

- Power off the left light: *"Q0.2 A7 Relay 1:2 A1" := false;*
"General"."Left sockets (Light)" := false;
- Power off the right light: *"Q0.3 A7 Relay 3:4 A1" := false;*
"General"."Right sockets (Light)" := false;
- Stop feeding the CO2: *"Q0.4 A7 Relay 13 A1" := false;*
"General"."CO2 valve" := false.

4.2.2 Automatic Mode

The automatic mode is the primary operational state for the climate chamber, designed to regulate environmental conditions based on predefined setpoints and schedules. This mode ensures that the climate chamber maintains optimal conditions for the growth and development of plants without requiring constant human intervention.

It is the main mode which is the default selection for the variable `Setpoints.mode`, so when PLC is powered and running the state is automatically selected.

In the automatic mode, the climate chamber's actuators and sensors operate based on the setpoints defined by the user. The system relies on a schedule to adjust these setpoints throughout the day, ensuring that the conditions within the chamber match the desired parameters at any given time. The following sections detail the various aspects of the automatic mode operation.

Humidity Control

The system manages humidity by comparing the current humidity level (`Sensors_data.Humidity`) with the target humidity setpoint (`Setpoints.Humidity_setpoint`). The control actions are determined based on thresholds defined around this setpoint:

- Humidifier activation: if the current humidity is below the lower threshold, the humidifier is turned on and the fan speed is set to maximum.
- Dehumidifier activation: if the current humidity is above the upper threshold, the dehumidifier is activated, the humidifier is turned off, and the fan is stopped.
- Neutral zone: if the humidity is within an inner threshold range around the setpoint, both the humidifier and dehumidifier are turned off, and the fan is stopped.
- Intermediate states: additional conditions handle intermediate states where the humidity is slightly above or below the inner threshold but not exceeding the upper or lower thresholds.

The humidifier and dehumidifier each have only two states – ON and OFF – as they are relay-controlled which may result in some fluctuations in humidity.

Temperature Control

The temperature control logic ensures the environment remains within a comfortable range by operating the air conditioning (AC) system in different modes:

- Heating mode: activated if the temperature drops below the lower threshold.
- Cooling mode: activated if the temperature rises above the upper threshold.
- Fan mode: activated if the temperature is within an inner threshold range around the setpoint, just circulating the air without heating or cooling.

- Mode adjustments: if the temperature is transitioning between inner and outer thresholds, the system may adjust the AC mode from heating or cooling to fan mode.

In order to prevent wear and tear on the air conditioning unit, inner and outer limits were introduced to prevent abrupt transitions between the heating and cooling modes. This technique not only increases the operational life of the equipment, but also ensures that the environmental parameters within the climatic chamber remain relatively constant over time.

This constant control of climatic conditions is very important when it comes to enhancing the vegetative and reproductive growth of plants. The system gives them care without temperature and humidity extremes with the aim of increasing the efficiency of the system's operation. This considerate strategy of urban farming practices finally results in healthier plants and higher yields of strawberries under cultivation in the city.

CO2 Control

Suspended ceilings are also equipped with systems which measure the level of the carbon dioxide concentration in the internal atmosphere and compare this level to a certain pre-established target value to maintain the content within minimum and maximum thresholds suitable for active growth of the plants. When the internal CO₂ is lower than the lower threshold within the chamber, the chamber CO₂ valve opens automatically to release more CO₂ which assists in the growth of the plants in the chamber.

For achieving the best adjustment of this feeding process, two timers are implemented. The first timer known as `Open_valve_timer` is used to determine the time in which the CO₂ valve is kept open and enables effective control of CO₂ concentration. The other timer which is referred to as `CO2_delay` helps in putting a time gap between the two consecutive feedings of CO₂. This avoids

the excessive feeding of CO₂ and also helps in reducing the oscillations of concentration level to a great extent so that even concentration is provided to the plants. To manage these parameters, favourable conditions that aid enhanced growth and better performance of urban agriculture can be achieved.

Light Control

The control unit capable of adjusting the light levels will control the lighting system as well, either by switching it on and off manually when required or operating on a timer. It adjusts the light sockets accordingly, watching over light intensity and duration limits. It guarantees the provision of the right light to plants during appropriate times.

Dew Point Management

The system will also monitor the internal chambers' temperature and ensure it is not close to the dew point to avoid the formation of water droplets. The system will switch on the dehumidifier and AC under the dry mode and switch off the humidifier when the temperature drops close to the dew point.

Setpoint Interpolation

In order to provide effective changes throughout the day, the temperature, humidity and CO₂ setpoints may be interpolated over a period of time using a timeline. The Value_from_schedule function is made for calculating these interpolated setpoints to allow smooth changes which are in line with the changes that normally occur in the environment.

In the automatic mode, the PLC is set to control the functioning of the climate chamber with respect to the time schedule and the setpoints assigned. The code of this mode is written in a way that all actuators would be given a command in case of any change in environmental conditions, so that safe and effective functioning of the system is achieved.

The automatic mode presents a full package of features for preserving the required conditions inside a climate chamber. Implementing a complex timetable and corresponding actuator control, the system is able to meet the different requirements of plants' growing processes with minimal intervention of manual correction. Such realization enables the climate chamber to be operated effectively and efficiently with respect to the urban agriculture experiments – resulting in the consistent and repeatable outcomes.

4.2.3 Manual Mode

In the manual mode, the control of the climate chamber's actuators is done directly through user-defined setpoints. This mode is made mainly for testing and manually adjusting the environmental conditions within the chamber at the time of maintenance.

Configuration of the Air Conditioner (AC)

In this mode, the air conditioner can be configured based on manual inputs. The following parameters are controlled:

- AC Power: The AC can be turned on or off.
- AC Mode: The mode of operation (cooling, heating, or fan) is selectable.
- Fan Speed: The speed of the fan can be set.
- Temperature Setpoint: The desired temperature can be specified.

Control of the Humidifier and Fan

The humidifier and its connected fan are also controlled manually. The system ensures that the humidifier does not operate if the dew point is close to the current temperature to avoid condensation issues.

Overview of the Manual Mode Control Logic

In the manual mode, the user has full control over the environmental settings.

This mode is vital for:

- Testing individual components: Ensuring each actuator and sensor works correctly.
- Making precise adjustments: Fine-tuning environmental conditions for specific requirements.
- Handling special conditions: Addressing scenarios where automatic control might not be suitable.

The manual settings provide a direct interface for users to influence the chamber's conditions without the predefined schedules and automated adjustments present in the automatic mode. This flexibility is important for experimental setups and troubleshooting.

In summary, the manual mode allows the user to control the climate chamber's environment directly, making for a hands-on approach to managing temperature, humidity, and other critical parameters.

4.3 Schedule

The importance of the scheduling mechanism in the control system of a climate chamber is essential for the automation of changing environmental parameters such as temperature, humidity, CO₂ or light. Within this system, predefined time slots are used ensuring the appropriateness of the chamber's interior environment to the plants' requirements at different times of the day.

The principal elements and capabilities of the scheduling system including its importance in sustaining favourable conditions for plants growth are presented below.

4.3.1 Scheduling Logic

The schedule implementation is based on a function block that handles arrays of times and corresponding values for each parameter. The function block evaluates the current time against the time slots and interpolates the values if necessary to provide the required setpoints.

Initially, a simple day/night state was requested for implementation, where the starting time defines when day setpoints are taken into account, and when the day end time expires, night values of setpoints are used in the control loop. However, for different types of plants, a more complex schedule might be required. Therefore, an extended version of the schedule was implemented.

In this extended schedule, a full day (24 hours) can be broken down into up to 12 time slots, with each subsequent time being in ascending order. Although the logic of the scheduler can accommodate 24 slots, only 12 slots were selected to ensure better readability and due to the limited space on the screen. The extended schedule is activated when the user increments the number of setpoints from the default value of 2 (which represents the simple day/night schedule) to any arbitrary value in the range of 3-12. This value can be changed in the UI on the Schedule screen (see Section 4.7.3).

When the extended schedule is activated, all the setpoints of the actuators are utilized (see Figure 4).

Number_of_setpoints_max	UInt
Number_of_setpoints	UInt
Current_time_index	UInt
▶ Times	Array[0..24] of Time_Of_Day
▶ Temperature_values	Array[0..24] of Real
▶ Humidity_values	Array[0..24] of Real
▶ CO2_values	Array[0..24] of Real
▶ Light_values	Array[0..24] of Real

Figure 4. Extended schedule data variables.

Figure 4 shows the "Times" array consisting of 24 values of type Time_Of_Day, which is formatted as hh:mm:ss. The following four lines represent setpoints for the air conditioner, humidifier, CO₂ injection and light controller relay.

4.3.2 Value_from_schedule Function Block

The Value_from_schedule function block is providing scheduled setpoints for various parameters. It uses the current time to determine the appropriate setpoint based on the predefined schedule. Key steps in this function block include the following:

1. Initialization: ensuring a value is always returned by setting an initial value.
2. Handling irregular schedules: managing day/night mode schedules when the start time is greater than the end time, handling cases where the current time falls between or outside the specified times.
3. Time index calculation: for regular schedules with more than two time slots, the function calculates the appropriate time index based on the current time.
4. Interpolation: if interpolation is enabled, the function interpolates between the start and end values for the current time slot to provide a smooth transition (see Section 4.3.3).

4.3.3 Interpolate Function Block

The Interpolate function block calculates the interpolated value for a parameter based on the elapsed time within the current time slot. The key steps include the following:

1. Delta calculations: calculating the differences between start and end times and values.

2. Elapsed time calculation: adjusting the elapsed time for cases where the end time is earlier than the start time.
3. Interpolation logic: computing the interpolated value based on the fraction of the elapsed time relative to the total time difference.

This scheduling mechanism ensures that the climate chamber can dynamically adjust its parameters to meet the specific requirements of the controlled environment, thereby enhancing the accuracy and efficiency of the system.

4.4 Implementing Modbus Communication in PLC

Modbus is a communication protocol widely used in industrial automation systems. It facilitates communication between devices over serial lines or Ethernet. The protocol operates in a master-slave configuration, where the master device initiates transactions (queries), and the slave devices respond.

In the project, Modbus communication was implemented between PLC, AC and dehumidifier. Implementing Modbus communication involves several steps.

4.4.1 Setting up and Read and Write Operations

For the Modbus cyclic interrupt with a cycle of 500 ms written in the LAD language was created. This language has a good graphical representation of all inputs and outputs of the function, so it is easier to configure and debug Modbus instances.

In order to get started with Modbus, it has to be configured using "MB_COMM_LOAD" instruction (see Figure 5). It is a one-time setup that must be completed before any read or write operations can take place. Upon completion of the configuration, the port becomes available for use by the "MB_MASTER" and "MB_SLAVE" instructions.

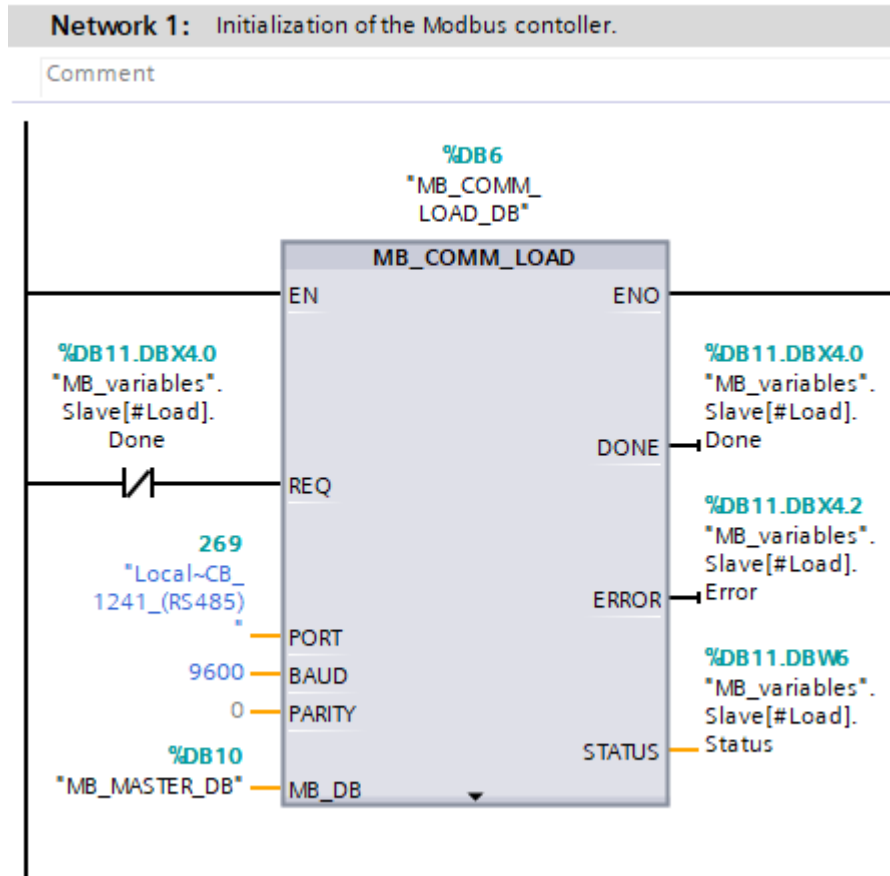


Figure 5. Initialization of Modbus communication

In Figure 5, the instruction's configuration inputs are located on the left side and the outputs are on the right side of the block. The inputs are the following:

- The enable input (EN) is a fundamental component of the LAD instruction within the EN/ENO mechanism. The execution of a function block is triggered only when the "EN" input holds a signal state of "1". Upon successful processing, the enable output (ENO) also assumes a signal state of "1". However, if an error occurs during execution, the "ENO" output is reset.
- REQ refers to the execution of the instruction upon a rising edge. As it was stated earlier, "MB_COMM_LOAD" must be called once, so this input will be true only at the first call, when its output "DONE" is not yet true, ensuring instruction is called only once.

- PORT refers to the ID of the communications port of a communication board. The instance is bind to this port once configured.
- BAUD is the baud rate selection. 9600 bps is configured on the hardware module and in the instruction correspondingly.
- PARITY – no parity is configured on the hardware module and in the instruction correspondingly.
- MB_DB serves as a reference to the instance data block pertaining to the "MB_MASTER" instructions. In the following networks, the "MB_MASTER" instance is used for communication with AC and the dehumidifier.

Outputs provide information about state of the initialization:

- ENO: as explained above it indicates if the execution was successful with no errors.
- DONE: Indicates that the execution of the instruction has been completed successfully, without any errors.
- ERROR: When true, signifies that an error has been identified, with an associated error code provided in the STATUS parameter.
- STATUS: Represents the error code related to port configuration. This code is shown on the screen Errors (see Section 4.7.4).

Upon successful completion of initialization without errors, the "MB_MASTER" instance, specified as the MB_DB input in the "MB_COMM_LOAD" instruction, becomes available for accessing data from one or more Modbus slave devices.

An example of the Modbus master is shown in Figure 6.

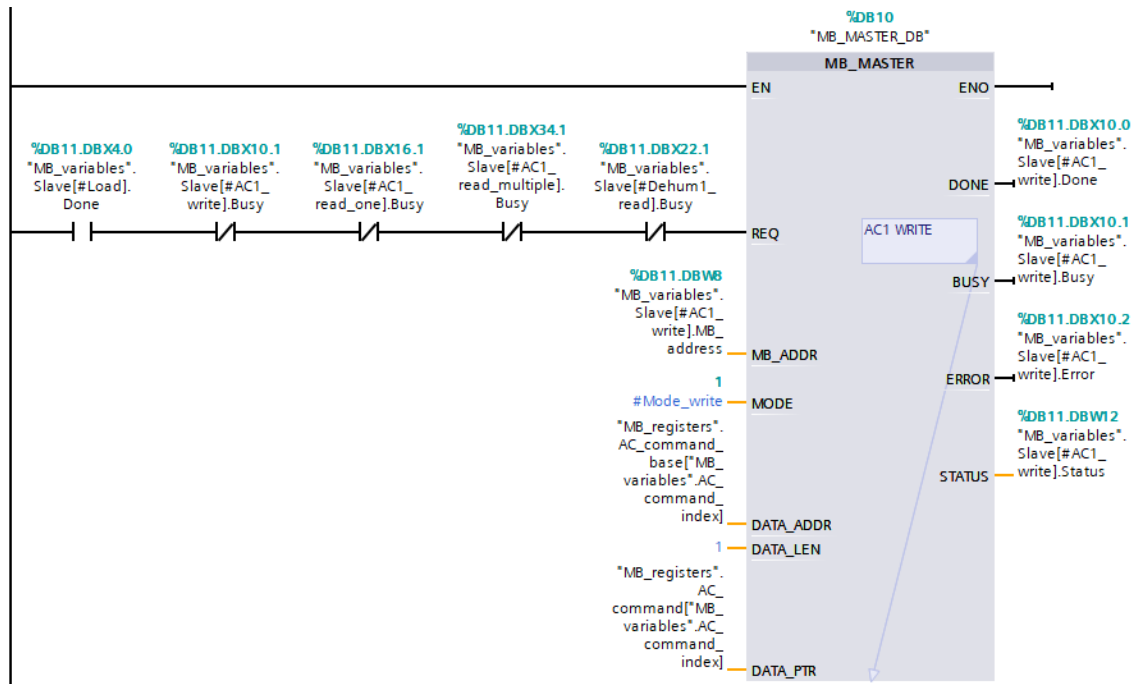


Figure 6. Modbus write operation

In Figure 6, MB_MASTER_DB instruction's inputs are located on the left side and the outputs on the right side of the block.

The outputs are similar to those of the initialization block, with the addition of the "BUSY" status, which indicates whether an "MB_MASTER" transaction is currently in progress. The state of "MB_MASTER" is stored in the array of structures named "MB_variables.Slave[]".

The inputs to the master are different, as described below:

- REQ: due to the fact that the Modbus instance is bound to the hardware port, MB_MASTER can be used only when no other instances are busy. For this reason, several checks are inserted.
- MB_ADDR: Modbus slave address. AC – 7, dehumidifier – 1.
- MODE: refers to the selection of operational mode, which delineates the type of request being made: read, write, or diagnostics. In this context, only the read (0) and write (1) modes are utilized.

- DATA_ADDR: indicates the initial address within the slave device, specifying the starting point for data access in the Modbus slave (see Section 4.4.2).
- DATA_LEN: data length, which specifies the number of bits or words to be accessed in this request.
- DATA_PTR: refers to the pointer that indicates the database or the specific memory address within the CPU where data is to be either written or retrieved.

Once MB_MASTER_DB instance is created with the LOAD_DB instruction, it is called three times: to send commands to AC (write operation) and two times to get (read operation) the status of the devices. In one read operation logic requests first 100 registers in AC (concatenated block of memory) in one operation which is effectively one cycle of a cyclic interrupt. Second instance reads 40 status registers in a loop (one by one) in dehumidifier. AC registers are placed back-to-back in a memory starting from Modbus address 400001, so multiple read operation can be used in this case. The dehumidifier, on other hand, has less uniform register addresses and they are saved in an array "DeHum_regs_addresses" with sequential read in a loop.

Two more Modbus operations can be used in a future development upon needs (they were implemented, but not used at the time of implementation):

- Sequential read of AC memory by one register per one Modbus cycle (500 ms) – if specific registers are needed aside from the first 100 "AC_read_selected", the array can serve this purpose.
- Commands (write operation) to the dehumidifier.

4.4.2 Modbus Data Variables

In order to get proper Modbus communication between PLC (master) and actuators (slaves), proper synchronization of Modbus instances is required. As it was mentioned earlier, only one instance (operation) at a time can be run, so the status of every request is saved in the data block for frequent usage.

Two data blocks were used in the project:

- **MB_registers:** this array is responsible for storing the addresses and values of the registers associated with both the AC and dehumidifier. The “MB_registers” data block plays a critical role in storage and retrieval of data during Modbus operations. When a read or write operation is performed, the corresponding data is either written to or read from the appropriate variable (array) withing this data block.
- **MB_variables:** the data block is utilized to manage the status of the Modbus instances, including monitoring whether an instance is busy, whether it has completed its operation, or whether it has encountered an error. This data block also contains indexes that are used to loop through arrays of register addresses and their values. By managing these statuses and indexes, the MB_variables data block ensures that only one Modbus operation is executed at any given time, thereby preventing conflicts and ensuring the smooth execution of communication tasks.

The structured use of these data blocks is instrumental in the effective management of Modbus communication within the climate chamber control system, enabling precise control and monitoring of the connected devices.

4.5 Error Handling

In the register’s range of the heat pump, there is a part responsible for alarms and errors indicating. For example, if device register number 10 holds 1, this indicates an alarm condition and then register number 11 holds an error code.

4.6 Safety Measures

Growing plants requires high attention to climate conditions as different types of plants are sensitive to variations of temperature, humidity, CO₂ and light levels. In order to ensure humidity does not exceed rapidly, the setpoint and moisture is not accumulated on the floor level, and distributed evenly, the humidifier fan shall be started at the same time with the humidifier. For the manual mode, when the humidifier relay (A2 Relay 2 A1) is energized, the fan is started automatically with the minimum possible speed.

The mode changes to OFF state if the schedule is invalid. An alarm with the text “Can't be started, fix schedule first” is shown under the main switch button on the home screen in UI.

The risk of condensation which may be harmful for the electrical components of the actuators is eliminated by measuring the dew point in every PLC cycle. If the current temperature is closer to the calculated dew point value than the configured dew point threshold (at the time of writing it was set as 2), then the following logic takes precedence over other control commands (except in the OFF mode):

- The humidifier is set off
- The dehumidifier is set on
- The air conditioner is set to mode “dry”

As specified in Section 3.4, the dehumidifier has operation limits which are measured in every PLC cycle. In case of violations, a special flag is set, and the dehumidifier cannot be started. On the home screen in the UI, the text “Not ready” is displayed next to the dehumidifier relay (A2 Relay 1 A1).

4.7 User Interface

The User interface is created on the HMI, which is not actual an Siemens touch panel HMI, but runtime software for PC-based visualization running on SIMATIC WinCC Runtime Advanced (see Figure 7).

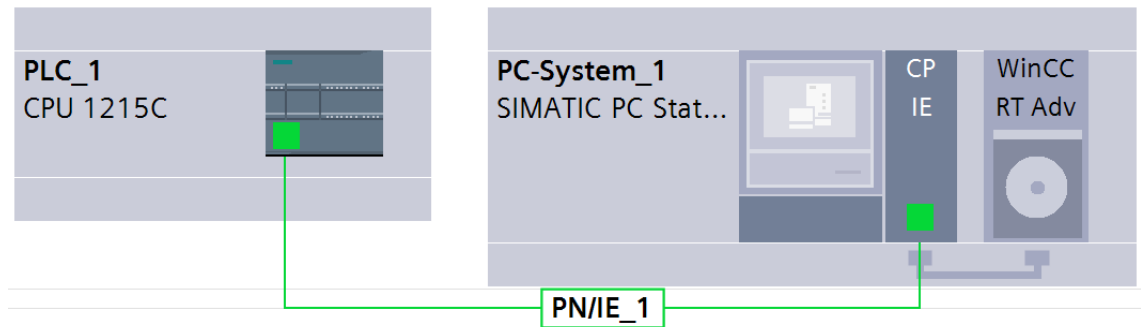


Figure 7. Devices and networks configuration

WinCC RT Advanced is the high-performance visualization software running on the developer's PC which is connected with the PLC via an Ethernet cable (PN/IE_1 in Figure 7).

The UI is the main point to access sensor readings and the change state of the actuators. It is linked to the project and connected with the PLC via TIA Portal IDE and is capable of changing the values of the variables in the runtime.

4.7.1 Screen Home

The main screen is selected by default. It has several sections enabling control over the whole system:

1. The "Climate control" section consists of one button (ON/OFF) and another button (Check schedule) that appears only if the schedule contains discrepancies, as shown in Figure 9 in Section 4.7.5.1.
2. The "Mode" section only has one button – the mode selector.

3. The “State” section, on other hand, has many buttons by means of which one can change the state of all relays and start or power off any individual actuator. The buttons are active in the manual mode only. In other modes, the buttons are representing the current state of the relays.
4. Section “Status” has information about the readings of the current sensors and the light status (the left and right parts of the chamber) – the yellow-coloured circle means that the light is on, whereas black means that the light is off. The sign “Interpolation ON” is shown only if interpolation is enabled.
5. All setpoints can be set in the “Setpoints” section on the “Home” screen. It has different visible elements depending on the amount of time slots selected in the schedule. Carbon dioxide feeding time and delay between feedings are common settings, which are always visible. If simple day/night schedule is selected, then the operator can set times and start and end (day and night) values to all actuators. When an extended schedule is used, then the “Check schedule” button appears, and it leads the operator to the Schedule screen.

4.7.2 Screen State

On this screen, the operator can analyse temperature, humidity and carbon dioxide trends graphically. The current state of the whole system is also presented on the screen – the state of the corresponding actuator is located below each trend graph. The “Calculations” section on the screen contains real time calculations of three vapour state conditions – dew point temperature, vapor pressure deficit and the third parameter required in the conditions control (on the screen, it is called VMD, or vapour mass deficit). The dew point temperature is used by inner logic. This is explained in Section 4.6.

4.7.3 Screen Schedule

The Schedule screen allows an operator to break down the 24 hours of a day into up to 12 parts each of which can have individual setpoints. The number of slots is set by incrementing or decreasing the value of the variable `Number_of_setpoints` (see Figure 4). A new slot appears once the variable is incremented by one. Every new line will hold recently used or default values if the slot was never used.

The right-hand side time and setpoints are representing the currently active time slot and its corresponding setpoints.

4.7.4 Screen Errors

The Errors screen handles errors that may happen in the Modbus protocol itself or the errors or service notifications of an AC and alarms related to the PLC operation itself.

On the left side of the screen, up to 10 unique Modbus errors may be shown. Every line of custom-made Modbus errors has the following information:

- Device and its operation (or Modbus instances) – six possible options such as:
 - “Empty”
 - “Modbus initialization”
 - “AC_1_write”
 - “AC_1_read”
 - “Dehum_1_read”
 - “Dehum_1_write”
- Error code – hexadecimal representation of the error messages of the “MB_MASTER” instruction
- Error list –description of errors

In the section Alarms (on the right side of the screen), other alarms and errors may exist in the build in the Alarm view or the window of the HMI device. The same Alarm view is available as a global screen which can be activated (popped up) with the keyboard button F12.

4.7.5 User Input Validation and Error Prevention

When dealing with user inputs, especially in a climate chamber control system, it is important to ensure that all inputs are valid and within the expected parameters. There are several measures done in terms of user input validation in this kind of a situation.

4.7.5.1 Schedule Time Discrepancies

The implemented schedule explained in Section 4.3 is designed in such a way that if the number of time slots is more than 2, then the time of every following line is greater than the time of the previous line. If this rule is violated, then an indication is given to the user. In Figure 8, the last time is equal to the previous, so an “Invalid schedule” warning in the bottom right corner is shown to the operator.

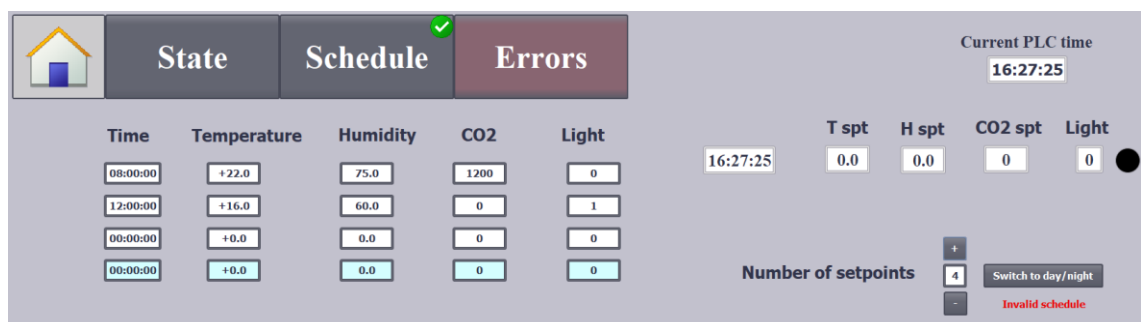


Figure 8. Invalid schedule. Warning message.

As a matter of fact, if the system does not have a valid schedule, the start of the actuators is prohibited (see Figure 9 below).

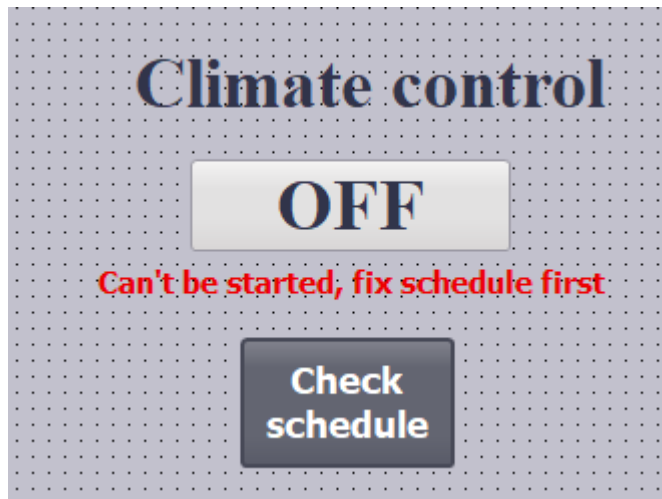


Figure 9. Invalid schedule. Mode change to manual or automatic is prohibited.

A red coloured warning, “Can’t be started, fix schedule first”, can be seen in Figure 9. The warning gives a hint on the needed action. The alarm is produced at the same time, and it should be acknowledged in the errors screen.

4.7.5.2 Out-of-range values

Every value entered by the user is validated against the allowed ranges for the specific parameter. For example, variable named "Setpoints".Number_of_setpoints defines the number of time slots set by the user. The value of this variable must lie within the range of 2 to the value set in variable named "Setpoints".Number_of_setpoints_max (set as 12 at the time of writing), so the value is limited by these two numbers. If the user attempts to input a value, such as 13, a warning message is displayed.

Validation is implemented using the internal mechanisms of TIA Portal through the properties of the variable.

A similar approach was implemented for all other variables exposed to a user input.

4.7.5.3 Invalid characters input

Invalid character inputs can lead to system errors, unexpected behaviour, and potentially hazardous situations. In the Tia Portal, the type of the variable linked to a I/O field (element used for a user input) defines the accepted type of an input. As a result, no additional validation is required.

5 Future Enhancements and Recommendations

As the climate chamber control system continues to evolve, there are several areas where future enhancements could be implemented to improve functionality, efficiency and reliability. This section outlines potential improvements and recommendations for the system.

5.1 Data Logging

Data logging is essential for monitoring and analysing the performance of the climate chamber control system. Initially, logging was performed on the connected PC's hard drive due to the unavailability of a special Siemens memory card. However, this PC is managed by Metropolia's administrative group and is subject to maintenance and updates, which could lead to data loss. To ensure data integrity and availability, it is recommended to use the PLC's internal storage for logging. The TIA Portal provides several functions specifically designed for logging data on internal PLC storage. These functions are shown in Figure 10.

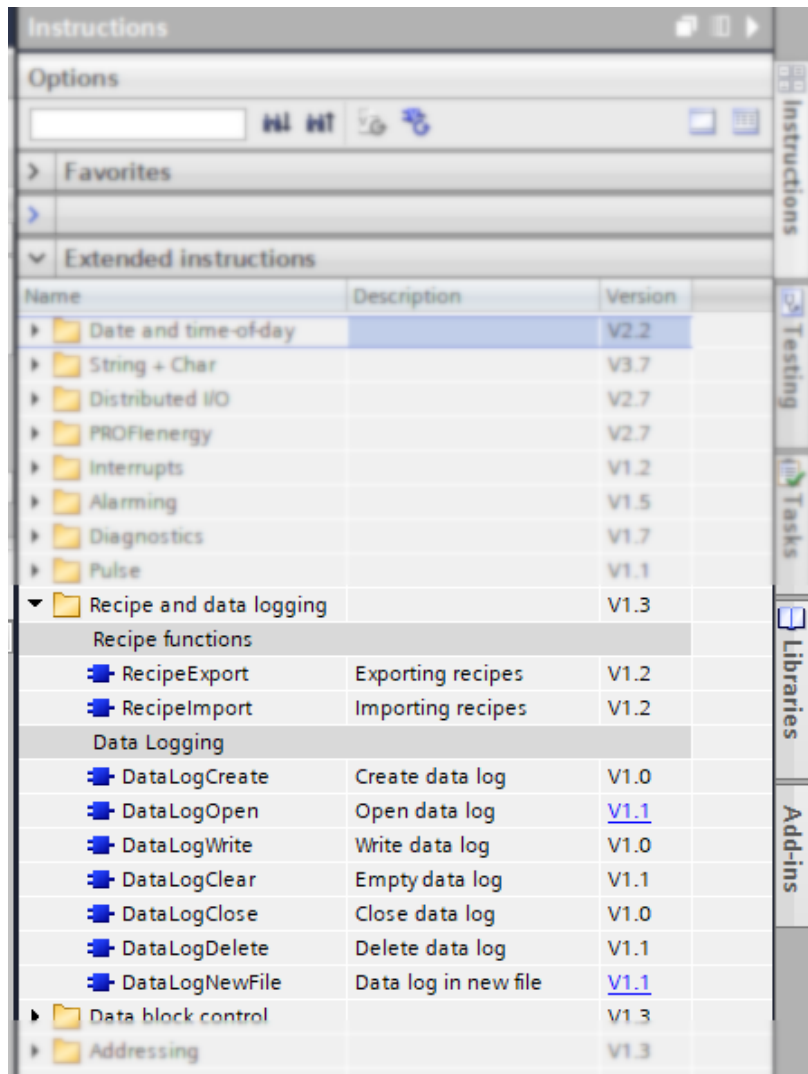


Figure 10. Recipe and data logging instruction set in TIA Portal.

As can be seen in Figure 10, there are instructions for all basic operations with log files, making maintenance easy.

Using the PLC's internal storage will enhance the reliability and stability of the data logging process, minimizing the risk of data loss during PC maintenance or updates.

5.2 Time Synchronization

Accurate time synchronization is crucial for the climate chamber control system, ensuring that all scheduled tasks and logged data are correctly timestamped.

According to the PLC datasheet [5], the maximum clock accuracy deviation is ± 60 s/month at 25 °C. To achieve a higher degree of accuracy and minimize the need for manual time synchronization, setting up an NTP server is recommended.

5.3 Runtime Errors

During the development stage, a few persistent errors were identified but not resolved due to their low priority and insignificant impact on the system's performance.

5.3.1 Air Conditioner Communication Error

At a certain point of the project, error number 65535 (-1) was registered in the heat pump register number 11, indicating a communication error between the INMBSPAN001R000 interface and the AC unit. The EIA-485 termination resistor is enabled in the INMBSPAN001R000 interface. Implementing a fail-safe biasing mechanism is recommended to address this issue.

5.3.2 Too Many Tags

An error message stating, "License key not available! Too many tags (Powertags) have been configured," appears every 5-10 minutes. This issue arose because a student license of TIA Portal v16 was used for development. The error is due to the license for WinCC Runtime Advanced either not being included in the student package or the license being insufficient. A business license, which is planned for future use, should resolve this issue.

6 Conclusion

This thesis presents the development and implementation of an advanced climate chamber control system using PLC. The primary goal was to design a reliable, efficient, and scalable system capable of maintaining precise

environmental conditions within a climate chamber, which is essential for various industrial and research applications.

The project employed a careful, reasonable and detailed approach to designing the climate chamber in a way that allows for maximum functionality while minimizing the need for manual input. Central to the climate control system is a Siemens PLC, well known for its reliability and ability to handle complex automation tasks. With the addition of sensors and actuators networked via the Modbus protocol and simple analogue lines, it was possible to achieve regulation of the temperature, humidity, CO₂, and light levels within a specified range.

The system has two modes of operation, i.e., automatic and manual, which provides the user with different control options. It means that the system can take charge of the operations while some operations can be manually carried out if the need arises. It has a scheduling function with 12 time slots to allow the environmental chamber to be adjusted according to different species of plants at different times of the day. Besides, the system is designed to be equipped with additional safety feature opportunities for error recovery. An ergonomic interface is also designed, and it is intended to be used with a view of achieving effective performance and usability.

One of the most important milestones of this project was the implementation of Modbus communication, which made it possible to organize quality data exchange between the PLC and other devices such as air conditioners or dehumidifiers. This configuration made it possible for the system to respond immediately to environmental changes within and outside the system itself, for the benefit of the plants' growth performance.

The system has been in operation for a few months without major problems; however, the final year project has outlined the possibilities of future enhancements. Improvements such as enhanced data storage and retrieval components, improved clocks for time alignment, and handling of errors that occur when the system is in use have been discussed. These recommendations

provide a scope for further advancement of the system with an aim of making sure that the system incorporates the latest urban agriculture technology.

In summary, this thesis makes a meaningful contribution to the piloting climate chamber automation, presenting the effectiveness of PLC based control systems in controlled environments for agricultural research. The achievements of this work address and extend the Metropolia UrbanFarmLab initiative as well as contribute to the development of sustainable food systems.

References

- 1 Luis Duarte, Marisol Osorio, Carlos A. Hincapie, Jorge A. Cardona-Gil. Prototype of a urban farming irrigation control lab. 2019 IEEE 4th Colombian Conference on Automatic Control (CCAC), October 15-18, 2019. Diez Hotel, Medellin, Colombia.
- 2 UrbanFarmLab, Collaboration Platforms in Research, Development and Innovation section of Metropolia web resource. [Online]. 2024. Accessed on 10 May 2024. Available from:
<https://www.metropolia.fi/en/rdi/collaboration-platforms/urbanfarmlab>
- 3 Cui Hui, Jin Hong, Loudon Rodney. An Overview of the Security of Programmable Logic Controllers in Industrial Control Systems. [Online]. 2024. Accessed on 10 May 2024. Available from:
<https://www.proquest.com/scholarly-journals/overview-security-programmable-logic-controllers/docview/3072306767/se-2?accountid=11363>
- 4 Robert C. Martin. Clean Code. A Handbook of Agile Software Craftsmanship. 2009. Addison Wesley.
- 5 Datasheet: SIMATIC S7-1200, CPU 1215C, compact CPU. [Online]. 2023. Accessed on 10 November 2023. Available from:
<https://support.industry.siemens.com/cs/pd/142915?pdtd=td&dl=en&lc=en-El>
- 6 Datasheet: Indoor Air Quality Sensors QPA10., QPA20... [Online]. 2023. Accessed on 09 November 2023. Available from:
<https://hit.sbt.siemens.com/RWD/modules/kernel/UI/slow/GetBinData.aspx?DTP=Data+Sheet+for+Product&SID=A6V11526870&EXT=.pdf&VALUE=Assets%5cA6V11526870+Indoor%2520Air%2520Quality%2520Sensors%2520QPA10..%2520QPA20..+en.pdf&KEY=3&RT=1699519514775>
- 7 User Manual: Air Conditioner and Modbus Communication Interface [Online]. 2024. Accessed on 1 February 2024. Available from:
<https://www.intesis.com/docs/user-manual-inmbspan001r000>
- 8 Operation Manual: Condensing Dehumidifier [Online]. 2022. Accessed on 1 February 2024. Available from:
<https://www.condair.hu/m/0/condair-dc-c-en-manual.pdf>